

TuniTech Abroad - Data Flow / Sequence Diagrams v1

1. Document Purpose

This document defines the main product data flows and system sequence diagrams for TuniTech Abroad MVP.

It describes how users, Django views, services, Celery tasks, PostgreSQL, Redis, France Travail API, OpenRouter, and email services interact.

This document comes after:

- BRD v1
- PRD v1
- Technical Architecture v1
- Database Schema v1
- Service Contracts v1

It comes before:

- Page + URL + View Map v1
- Implementation Roadmap v1
- MVP Backlog v1
- Repository Setup Plan v1
- Coding Phase 0

2. System Actors

Human Actors

- Anonymous visitor
- Authenticated user
- Admin user

Internal System Components

- Django Web App
- Django Admin
- PostgreSQL
- Redis
- Celery Worker
- Celery Beat

External Services

- France Travail API
- OpenRouter API
- Email provider
- Google OAuth
- GitHub OAuth

3. Core Data Flow Rule

The system follows this rule:

- Views receive requests.
- Views validate basic access.
- Views call services.
- Services execute business logic.
- Services persist data.
- Celery tasks call services.
- Models store data only.

Forbidden:

- No external API calls from models.
- No LLM calls from views.
- No matching logic inside templates.
- No business logic directly in Celery tasks.
- No live France Travail API calls for normal user search.

4. Signup and Onboarding Flow

4.1 Purpose

Create a user account, provision profile data, and prepare the user for CV upload/profile completion.

4.2 Flow

User
-> opens register/login page
-> chooses Google, GitHub, or email/password
-> django-allauth authenticates user
-> AccountProvisioningService creates profile and preferences
-> user is redirected to onboarding/profile completion

4.3 Sequence Diagram

```
sequenceDiagram
    actor User
    participant Web as Django Web App
    participant Allauth as django-allauth
    participant OAuth as Google/GitHub OAuth
    participant APS as AccountProvisioningService
    participant DB as PostgreSQL
    participant Analytics as UserEventService

    User->>Web: Open login/register
    User->>Web: Choose Google/GitHub/email
    Web->>Allauth: Start authentication

    alt Google or GitHub
        Allauth->>OAuth: OAuth authentication
        OAuth-->>Allauth: Verified user data
    else Email/password
        Allauth->>DB: Create local user
        Allauth->>User: Send verification email
    end

    Allauth->>DB: Create or retrieve User
    Web->>APS: provision_new_user(user, provider)
    APS->>DB: Create CandidateProfile if missing
    APS->>DB: Create EmailPreference if missing
    APS->>Analytics: record(signup_completed)
    Web-->>User: Redirect to onboarding/profile
```

4.4 Data Written

User
CandidateProfile
EmailPreference
UserEvent
SocialAccount, if OAuth

4.5 Failure Handling

OAuth failure -> show login error.
Email not verified -> restrict email-dependent features.
Duplicate verified email -> link social account if safe.

5. France Travail Job Ingestion Flow

5.1 Purpose

Fetch France IT jobs from France Travail API into local storage.

The public job search must read from PostgreSQL, not call France Travail live.

5.2 Flow

Celery Beat
-> triggers scheduled ingestion
-> JobIngestionService creates IngestionRun
-> FranceTravailClient calls France Travail API
-> raw payload saved in RawJobRecord

-> changed/new records enqueue normalization
-> stale/expired jobs updated

5.3 Sequence Diagram

```
sequenceDiagram
    participant Beat as Celery Beat
    participant Worker as Celery Worker
    participant Ingestion as JobIngestionService
    participant FT as FranceTravailClient
    participant API as France Travail API
    participant DB as PostgreSQL
    participant Redis as Redis/Celery Queue

    Beat->>Worker: sync_france_travail_jobs()
    Worker->>Ingestion: sync_source("france_travail")
    Ingestion->>DB: Create IngestionRun(running)
    Ingestion->>FT: search_offers(params)
    FT->>API: HTTP search request
    API-->>FT: Raw offers response
    FT-->>Ingestion: Raw response

    loop For each offer
        Ingestion->>DB: compute source_job_id + payload_hash
        alt New or changed job
            Ingestion->>DB: create/update RawJobRecord
            Ingestion->>Redis: enqueue normalize_raw_job_record(raw_id)
        else Unchanged job
            Ingestion->>DB: update last_seen_at
        end
    end

    Ingestion->>DB: mark stale/expired jobs
    Ingestion->>DB: Update IngestionRun(success/partial_success)
```

5.4 Data Written

IngestionRun
RawJobRecord
NormalizedJob.status, for stale/expired updates

5.5 Failure Handling

France Travail timeout -> retry with backoff.
Rate limit -> pause/retry safely.
Partial page failure -> IngestionRun.partial_success.
Full failure -> IngestionRun.failed.
Raw payload is never discarded if fetched successfully.

6. Job Normalization Flow

6.1 Purpose

Convert raw France Travail records into searchable, normalized job records.

6.2 Flow

Celery task receives RawJobRecord ID
-> JobNormalizationService reads raw payload
-> maps source fields defensively
-> classifies job type, remote type, experience, language
-> creates/updates NormalizedJob
-> updates PostgreSQL search_vector
-> enqueues job skill extraction

6.3 Sequence Diagram

```
sequenceDiagram
    participant Worker as Celery Worker
    participant Norm as JobNormalizationService
    participant Classifier as JobClassificationService
    participant DB as PostgreSQL
    participant Redis as Redis/Celery Queue

    Worker->>DB: Load RawJobRecord
    Worker->>Norm: normalize(raw_record)
```

```

Norm->>DB: Read raw_payload_json
Norm->>Classifier: classify(payload, description, title)
Classifier-->>Norm: job_type, remote_type, experience, languages
Norm->>DB: Create/update NormalizedJob
Norm->>DB: Update search_vector
Norm->>DB: RawJobRecord.normalization_status=success
Norm->>Redis: enqueue extract_job_skills(job_id)

```

6.4 Data Written

```

NormalizedJob
RawJobRecord.normalization_status
RawJobRecord.normalization_error, if failed

```

6.5 Failure Handling

Missing source field -> set field unknown/null, do not crash.
 Unusable payload -> normalization_status=failed.
 Failed raw record remains available for admin re-normalization.

7. Job Skill Extraction Flow

7.1 Purpose

Extract canonical skills from normalized jobs and attach them to the job for matching.

7.2 Flow

```

Celery task receives NormalizedJob ID
-> rule-based extraction finds skill candidates
-> optional LLM extraction runs if needed
-> SkillNormalizerService maps aliases to canonical skills
-> NormalizedJobSkill rows are created/updated
-> unknown terms go to UnmatchedSkillCandidate
-> job search_vector is rebuilt

```

7.3 Sequence Diagram

```

sequenceDiagram
    participant Worker as Celery Worker
    participant JSE as JobSkillExtractionService
    participant LLM as PromptRunnerService
    participant SNS as SkillNormalizerService
    participant DB as PostgreSQL

    Worker->>DB: Load NormalizedJob
    Worker->>JSE: extract_for_job(job)
    JSE->>JSE: Rule-based skill candidate extraction

    alt Rule extraction weak or configured for LLM
        JSE->>LLM: run(job_skill_extraction, job text)
        LLM-->>JSE: Raw skill candidates
    end

    JSE->>SNS: normalize_many(raw_skills, source_type="job", source_id=job.id)
    SNS->>DB: Find SkillAlias matches
    SNS->>DB: Create/update UnmatchedSkillCandidate for unknowns
    SNS-->>JSE: Canonical skills + unmatched candidates

    JSE->>DB: Upsert NormalizedJobSkill
    JSE->>DB: Update required_skills_json / optional_skills_json
    JSE->>DB: Update skill_extraction_status
    JSE->>DB: Rebuild search_vector

```

7.4 Data Written

```

NormalizedJobSkill
UnmatchedSkillCandidate
NormalizedJob.required_skills_json
NormalizedJob.optional_skills_json
NormalizedJob.skill_extraction_status
NormalizedJob.search_vector
LLMUsageLog, if LLM used
LLMCacheEntry, if LLM used

```

7.5 Failure Handling

LLM failure -> keep rule-based extraction if available.
No skills found -> skill_extraction_status=not_enough_text or failed.
Unknown skill text -> admin review queue, not auto-created skill.

8. Public Job Search Flow

8.1 Purpose

Allow anonymous and authenticated users to search active France IT jobs from local PostgreSQL.

8.2 Flow

User opens /jobs/
-> filters/search query submitted
-> JobSearchService uses PostgreSQL full-text search
-> active jobs returned
-> job search event recorded

8.3 Sequence Diagram

```
sequenceDiagram
    actor User
    participant Web as Django View
    participant Search as JobSearchService
    participant DB as PostgreSQL
    participant Analytics as UserEventService

    User->>Web: GET /jobs/?query=python
    Web->>Search: search(filters, user)
    Search->>DB: PostgreSQL full-text search on NormalizedJob.search_vector
    DB-->>Search: Paginated active jobs
    Search-->>Web: PaginatedResult
    Web->>Analytics: record(job_search)
    Web-->>User: Render job results
```

8.4 Data Read

NormalizedJob
NormalizedJobSkill
JobRecommendation, only for authenticated best-match sort

8.5 Data Written

UserEvent(job_search)

8.6 Failure Handling

No results -> show empty state.
Invalid filters -> ignore or show validation message.
Search must not call France Travail API.

9. Job Detail and High-Intent Revalidation Flow

9.1 Purpose

Show job details and optionally refresh old jobs before high-intent actions.

9.2 Flow

User opens job detail
-> JobQueryService loads job by public_id
-> if job is old, JobRevalidationService refreshes or enqueues refresh
-> job detail rendered

9.3 Sequence Diagram

```
sequenceDiagram
    actor User
    participant Web as Django View
```

```

participant Query as JobQueryService
participant Reval as JobRevalidationService
participant FT as FranceTravailClient
participant DB as PostgreSQL
participant Analytics as UserEventService

User->>Web: GET /jobs/<public_id>/
Web->>Query: get_public_job(public_id)
Query->>DB: Load NormalizedJob by public_id
DB-->>Query: Job
Query-->>Web: Job

alt job.last_fetched_at older than threshold
  Web->>Reval: revalidate_if_needed(job)
  Reval->>FT: get_offer_detail(source_job_id)
  FT-->>Reval: Fresh raw payload or error
  Reval->>DB: Update RawJobRecord/NormalizedJob/status if possible
end

Web->>Analytics: record(job_detail_view)
Web-->>User: Render job detail

```

9.4 Failure Handling

Revalidation failure -> render existing job with warning.
 Expired job -> show stale/expired label.
 Job not found -> 404.

10. Anonymous Quick Match Flow

10.1 Purpose

Reduce signup friction by allowing anonymous users to test a simple match without CV upload.

10.2 Flow

Anonymous user opens job
 -> clicks quick match
 -> enters 3-5 skills, experience level, French level
 -> QuickMatchService normalizes skills and calculates teaser score
 -> result shown
 -> full analysis gated behind signup/CV upload

10.3 Sequence Diagram

```

sequenceDiagram
  actor User
  participant Web as Django View
  participant JobQuery as JobQueryService
  participant Quick as QuickMatchService
  participant Skills as SkillNormalizerService
  participant DB as PostgreSQL
  participant Analytics as UserEventService

  User->>Web: POST /jobs/<public_id>/quick-match/
  Web->>JobQuery: get_public_job(public_id)
  JobQuery->>DB: Load job
  DB-->>JobQuery: NormalizedJob
  Web->>Quick: run_quick_match(session, job, skills, experience, french)
  Quick->>Skills: normalize_many(entered_skills, "quick_match")
  Skills->>DB: Match aliases / create unmatched candidates
  Skills-->>Quick: Canonical skills
  Quick->>Quick: Calculate estimated teaser score
  Quick->>DB: Create QuickMatchSession
  Quick->>Analytics: record(quick_match_completed)
  Web-->>User: Show teaser score + CTA to upload CV

```

10.4 Data Written

QuickMatchSession
 UnmatchedSkillCandidate, if unknown skill
 UserEvent(quick_match_completed)

10.5 Failure Handling

Too many attempts -> rate limit.

No valid skill entered -> validation message.
No LLM used.
No CV stored.
QuickMatchSession expires after 24h.

11. CV Upload and Parsing Flow

11.1 Purpose

Allow authenticated users to upload a PDF CV, extract structured profile data, and prepare profile confirmation.

11.2 Flow

User uploads PDF
-> CVUploadService validates and stores private file
-> parse_cv Celery task runs
-> text extraction
-> deterministic field extraction
-> LLM structured extraction
-> skill normalization
-> CVParsedData stored
-> CandidateProfile draft updated
-> user reviews/edits profile

11.3 Sequence Diagram

```
sequenceDiagram
    actor User
    participant Web as Django View
    participant Consent as ConsentService
    participant Upload as CVUploadService
    participant Redis as Redis/Celery Queue
    participant Worker as Celery Worker
    participant Parser as CVParsingService
    participant Text as CVTextExtractionService
    participant Exact as CVDeterministicExtractorService
    participant CVLLM as CVLLMExtractionService
    participant Skills as SkillNormalizerService
    participant Profile as ProfileCompletenessService
    participant DB as PostgreSQL

    User->>Web: POST /dashboard/cv/ with PDF
    Web->>Consent: record(cv_processing)
    Web->>Upload: upload_cv(user, uploaded_file)
    Upload->>DB: Deactivate previous active CV
    Upload->>DB: Create CVUpload(pending)
    Upload->>Redis: enqueue parse_cv(cv_upload_id)
    Web-->>User: Show processing state

    Worker->>DB: Load CVUpload via all_objects
    Worker->>Parser: parse(cv_upload)
    Parser->>DB: Set parse_status=processing
    Parser->>Text: extract_text(cv_upload)
    Text-->>Parser: raw_text or too_little_text

    alt Text readable
        Parser->>Exact: extract(raw_text)
        Exact-->>Parser: email/phone/links
        Parser->>CVLLM: extract_structured(cv_upload, raw_text)
        CVLLM-->>Parser: structured JSON or warning
        Parser->>Skills: normalize_many(extracted_skills, "cv", cv_id)
        Skills-->>Parser: canonical skills + unmatched
        Parser->>DB: Create/update CVParsedData
        Parser->>DB: Create/update ProfileSkill
        Parser->>DB: Prefill CandidateProfile draft fields
        Parser->>Profile: update_profile_score(profile)
        Parser->>DB: Set parse_status=parsed or parsed_with_warnings
    else Text unreadable
        Parser->>DB: Set parse_status=failed, reason=too_little_text
    end
end
```

11.4 Data Written

ConsentRecord
CVUpload
CVParsedData

```
ProfileSkill
CandidateProfile draft fields
UnmatchedSkillCandidate
LLMUsageLog
LLMCacheEntry
UserEvent(cv_uploaded/cv_parse_failed)
```

11.5 Failure Handling

Invalid file -> reject before saving.
File too large -> reject before saving.
Scanned/unreadable PDF -> parse failed, user can complete profile manually.
LLM failure -> parsed with warnings if deterministic extraction is usable.
Deleted CV -> parse task aborts.

12. Profile Confirmation Flow

12.1 Purpose

Ensure parsed CV data is reviewed by the user before becoming trusted matching data.

12.2 Flow

User opens profile page
-> sees extracted fields and missing fields
-> edits/confirm profile
-> ProfileUpdateService saves confirmed data
-> profile completeness recalculated
-> recommendations marked stale
-> recommendation refresh enqueued

12.3 Sequence Diagram

```
sequenceDiagram
    actor User
    participant Web as Django View
    participant Update as ProfileUpdateService
    participant Complete as ProfileCompletenessService
    participant Redis as Redis/Celery Queue
    participant DB as PostgreSQL
    participant Analytics as UserEventService

    User->>Web: POST /dashboard/profile/
    Web->>Update: update_profile(user, cleaned_data)
    Update->>DB: Update CandidateProfile
    Update->>DB: Update ProfileSkill if skills changed
    Update->>Complete: calculate/update score
    Complete->>DB: Save completeness score/status
    Update->>DB: Mark JobRecommendation stale
    Update->>Redis: enqueue refresh_user_recommendations(user, profile_updated)
    Update->>Analytics: record(profile_completed/profile_updated)
    Web-->>User: Show updated profile
```

12.4 Data Written

```
CandidateProfile
ProfileSkill
JobRecommendation.status=stale
UserEvent
```

13. Full Authenticated Match Flow

13.1 Purpose

Create a persisted match result between the authenticated user profile/CV and a selected job.

13.2 Flow

User opens job
-> clicks full match
-> MatchResultService validates profile/job
-> MatchScoringService calculates deterministic score

-> MatchResult saved with snapshots
-> optional LLM explanation can be requested

13.3 Sequence Diagram

```
sequenceDiagram
    actor User
    participant Web as Django View
    participant JobQuery as JobQueryService
    participant MatchResultSvc as MatchResultService
    participant Score as MatchScoringService
    participant DB as PostgreSQL
    participant Analytics as UserEventService

    User->>Web: POST /jobs/<public_id>/match/
    Web->>JobQuery: get_public_job(public_id)
    JobQuery->>DB: Load NormalizedJob
    DB-->>JobQuery: Job
    Web->>MatchResultSvc: create_match_result(user, job)
    MatchResultSvc->>DB: Load CandidateProfile + active CV
    MatchResultSvc->>Score: calculate(profile, job, cv)
    Score-->>MatchResultSvc: FitScoreResult
    MatchResultSvc->>DB: Save MatchResult with profile/job snapshots
    MatchResultSvc->>Analytics: record(match_generated)
    Web-->>User: Show match result
```

13.4 Data Written

```
MatchResult
UserEvent(match_generated)
```

13.5 Failure Handling

No usable profile -> redirect to profile completion.
No active CV but enough manual profile -> allow match with warning.
Expired job -> allow view with job_may_be_expired risk flag.

14. Match Explanation LLM Flow

14.1 Purpose

Generate optional explanation using OpenRouter without changing the deterministic score.

14.2 Flow

User requests explanation
-> MatchExplanationService builds structured payload
-> PromptRunnerService checks cache and rate limits
-> OpenRouter called if no cache
-> explanation validated
-> MatchResult updated

14.3 Sequence Diagram

```
sequenceDiagram
    actor User
    participant Web as Django View
    participant Explain as MatchExplanationService
    participant Prompt as PromptRunnerService
    participant Rate as LLMLimitService
    participant OR as OpenRouterClient
    participant API as OpenRouter API
    participant DB as PostgreSQL

    User->>Web: POST /matches/<public_id>/explain/
    Web->>Explain: generate(match_result, user)
    Explain->>Prompt: run(match_explanation, structured_payload, user, cache_key)
    Prompt->>DB: Check LLMCacheEntry
    alt Cache hit
        DB-->>Prompt: Cached response
    else Cache miss
        Prompt->>Rate: assert_allowed(user, match_explanation)
        Rate-->>Prompt: Allowed
        Prompt->>OR: chat_json(...)
        OR->>API: OpenRouter request
        API-->>OR: JSON/text response
```

```

    OR-->>Prompt: Response
    Prompt-->>DB: Save LLMCacheEntry
    Prompt-->>DB: Save LLMUsageLog
end
Prompt-->>Explain: LLMRunResult
Explain-->>DB: Update MatchResult explanation
Web-->>User: Show explanation

```

14.4 Guardrails

LLM cannot change fit_score.
 LLM cannot invent missing skills.
 LLM cannot promise employment.
 LLM cannot provide visa/legal advice.
 LLM gets structured score data, not free-form unchecked context.

15. Recommendation Refresh Flow

15.1 Purpose

Generate stored recommendations so dashboard loads quickly.

15.2 Flow

```

Trigger happens
-> RecommendationService creates RecommendationRun
-> active jobs are prefiltered
-> MatchScoringService calculates fit score
-> ranking_score calculated
-> JobRecommendation rows stored

```

15.3 Sequence Diagram

```

sequenceDiagram
    participant Trigger as View/Celery/Admin
    participant Reco as RecommendationService
    participant Score as MatchScoringService
    participant DB as PostgreSQL

    Trigger->>Reco: refresh_for_user(user, trigger_type)
    Reco->>DB: Create RecommendationRun(running)
    Reco->>DB: Load profile + active CV
    Reco->>DB: Prefilter active France jobs

    loop Candidate jobs
        Reco->>Score: calculate(profile, job, cv)
        Score-->>Reco: FitScoreResult
        Reco->>Reco: Calculate ranking_score
    end

    Reco->>DB: Mark old recommendations stale
    Reco->>DB: Upsert top JobRecommendation rows
    Reco->>DB: Complete RecommendationRun(success)

```

15.4 Triggers

CV uploaded/replaced
 Profile updated
 New jobs imported
 Dashboard recommendations stale
 Nightly refresh
 Admin manual refresh

15.5 Data Written

RecommendationRun
 JobRecommendation

15.6 Failure Handling

Profile incomplete -> RecommendationRun.failed or skipped.
 No jobs found -> success with zero recommendations.
 Worker retry should not create duplicates.

16. Dashboard Recommendation Query Flow

16.1 Purpose

Show recommendations without blocking the dashboard.

16.2 Flow

User opens recommendations page
-> RecommendationQueryService reads stored recommendations
-> if stale, returns current data and enqueues refresh
-> if none, enqueues generation and shows pending state

16.3 Sequence Diagram

```
sequenceDiagram
    actor User
    participant Web as Django View
    participant Query as RecommendationQueryService
    participant Redis as Redis/Celery Queue
    participant DB as PostgreSQL

    User->>Web: GET /dashboard/recommendations/
    Web->>Query: get_dashboard_recommendations(user)
    Query->>DB: Load active/stale recommendations

    alt Fresh recommendations exist
        Query-->>Web: Return recommendations
    else Stale recommendations exist
        Query->>Redis: enqueue refresh_user_recommendations
        Query-->>Web: Return stale recommendations with flag
    else No recommendations
        Query->>Redis: enqueue refresh_user_recommendations
        Query-->>Web: Return pending state
    end

    Web-->>User: Render dashboard
```

17. Saved Job Flow

17.1 Purpose

Allow authenticated users to save or remove jobs.

17.2 Sequence Diagram

```
sequenceDiagram
    actor User
    participant Web as Django View
    participant Saved as SavedJobService
    participant JobQuery as JobQueryService
    participant DB as PostgreSQL
    participant Analytics as UserEventService

    User->>Web: POST /jobs/<public_id>/save/
    Web->>Saved: save_job(user, public_id)
    Saved->>JobQuery: get_public_job(public_id)
    JobQuery->>DB: Load job
    Saved->>DB: get_or_create SavedJob(user, job)
    Saved->>Analytics: record(saved_job_created)
    Web-->>User: Saved job confirmation
```

17.3 Idempotency

Saving the same job twice returns existing SavedJob.

18. Weekly Digest Email Flow

18.1 Purpose

Send opt-in weekly job recommendation emails without duplicates.

18.2 Flow

```
Celery Beat triggers weekly digest
-> WeeklyDigestService creates EmailBatch
-> eligible users selected
-> top recommendations rendered
-> EmailSenderService sends email with idempotency key
```

18.3 Sequence Diagram

```
sequenceDiagram
    participant Beat as Celery Beat
    participant Worker as Celery Worker
    participant Digest as WeeklyDigestService
    participant Active as ActiveUserRecommendationService
    participant Email as EmailSenderService
    participant Provider as Email Provider
    participant DB as PostgreSQL

    Beat->>Worker: send_weekly_digest(period_start, period_end)
    Worker->>Digest: send_weekly_digest(...)
    Digest->>DB: Create/get EmailBatch
    Digest->>Active: get_active_users(14)
    Active->>DB: Query eligible active users
    Active-->>Digest: Users

    loop Eligible users
        Digest->>DB: Load top recommendations
        Digest->>Email: send(..., idempotency_key)
        Email->>DB: Check EmailEvent.idempotency_key
        alt Not sent before
            Email->>DB: Create EmailEvent(queued)
            Email->>Provider: Send email
            Provider-->>Email: Provider status
            Email->>DB: Update EmailEvent(sent/failed)
        else Already sent
            Email-->>Digest: Return existing EmailEvent
        end
    end

    Digest->>DB: Mark EmailBatch success/partial_success
```

18.4 Eligibility

```
verified email
weekly_digest_enabled = true
active within last 14 days
usable profile
new relevant recommendations
```

18.5 Idempotency Key

```
weekly_digest:{user_id}:{year}-W{week}
```

19. Account Deletion Flow

19.1 Purpose

Delete or anonymize user-owned personal data safely.

19.2 Flow

```
User requests account deletion
-> DeletionRequest created
-> user login disabled
-> Celery worker processes deletion
-> CV files removed
-> candidate data deleted
-> analytics anonymized
-> user deleted/anonymized
-> DeletionRequest completed
```

19.3 Sequence Diagram

```
sequenceDiagram
    actor User
```

```

participant Web as Django View
participant Delete as AccountDeletionService
participant Redis as Redis/Celery Queue
participant Worker as Celery Worker
participant CVDel as CVDeletionService
participant DB as PostgreSQL
participant Storage as Private File Storage

User->>Web: POST /dashboard/settings/delete-account/
Web->>Delete: request_deletion(user)
Delete->>DB: Create DeletionRequest(pending)
Delete->>DB: Disable user login
Delete->>Redis: enqueue process_account_deletion
Web-->>User: Deletion request accepted

Worker->>DB: Load DeletionRequest
Worker->>Delete: process_request(deletion_request)
Delete->>DB: Mark processing
Delete->>DB: Load user CVs via all_objects

loop CV files
    Delete->>Storage: Delete physical file
    Delete->>DB: Delete/clear CVParsedData
    Delete->>DB: Mark/delete CVUpload
end

Delete->>DB: Delete CandidateProfile + ProfileSkill
Delete->>DB: Delete SavedJob
Delete->>DB: Delete MatchResult
Delete->>DB: Delete JobRecommendation
Delete->>DB: Delete EmailPreference
Delete->>DB: Delete social accounts
Delete->>DB: Anonymize/delete UserEvent
Delete->>DB: Delete/anonymize User
Delete->>DB: Mark DeletionRequest completed

```

19.4 Failure Handling

Failure -> DeletionRequest.failed with error summary.
 Never leave user active if deletion is already processing.
 Admin can retry failed deletion request.

20. Admin Skill Review Flow

20.1 Purpose

Allow admin to map unknown skills into canonical taxonomy.

20.2 Sequence Diagram

```

sequenceDiagram
    actor Admin
    participant AdminUI as Django Admin
    participant Review as UnmatchedSkillReviewService
    participant DB as PostgreSQL

    Admin->>AdminUI: Select unmatched skill candidate
    Admin->>AdminUI: Map to canonical Skill
    AdminUI->>Review: map_candidate(candidate_id, skill_id, admin)
    Review->>DB: Update UnmatchedSkillCandidate(status=mapped)
    Review->>DB: Optionally create SkillAlias
    Review->>DB: Set reviewed_by/reviewed_at
    AdminUI-->>Admin: Mapping completed

```

20.3 Rule

LLM never creates new canonical skills automatically.
 Unknown extracted skills go to admin review.

21. Admin Reprocessing Flow

21.1 Purpose

Allow admin to re-run normalization, skill extraction, CV parsing, or recommendations using service layer.

21.2 Sequence Diagram

```
sequenceDiagram
    actor Admin
    participant AdminUI as Django Admin
    participant Service as Domain Service
    participant Redis as Redis/Celery Queue
    participant DB as PostgreSQL

    Admin->>AdminUI: Select records + action
    AdminUI->>Service: Validate admin action

    alt Immediate safe action
        Service->>DB: Update records
    else Heavy action
        Service->>Redis: Enqueue background task
    end

    AdminUI-->>Admin: Action accepted
```

21.3 Supported Admin Actions

- Re-normalize RawJobRecord
- Re-extract job skills
- Map unmatched skills
- Refresh user recommendations
- Re-parse CV, if not deleted
- Retry failed ingestion

22. Health Check Flow

22.1 Purpose

Expose system health for deployment and manual checks.

22.2 Sequence Diagram

```
sequenceDiagram
    actor Probe
    participant Web as Django Web App
    participant Health as HealthCheckService
    participant DB as PostgreSQL
    participant Redis as Redis

    Probe->>Web: GET /health/
    Web->>Health: check()
    Health->>DB: Lightweight DB check
    Health->>Redis: Lightweight Redis ping
    Health-->>Web: status JSON
    Web-->>Probe: 200 ok or degraded response
```

22.3 Output

```
{
  "status": "ok",
  "database": "ok",
  "redis": "ok"
}
```

23. End-to-End MVP Data Lifecycle

23.1 Job Data Lifecycle

- France Travail API
- > RawJobRecord
- > NormalizedJob
- > NormalizedJobSkill
- > PostgreSQL search_vector
- > JobSearchService
- > MatchScoringService
- > JobRecommendation

-> WeeklyDigestService

23.2 Candidate Data Lifecycle

User signup
-> CandidateProfile created
-> CVUpload
-> CVParsedData
-> ProfileSkill
-> User confirms profile
-> MatchResult
-> JobRecommendation
-> SavedJob
-> Account deletion removes/anonymizes user-owned data

23.3 LLM Data Lifecycle

PromptVersion
-> PromptRunnerService
-> LLMCacheEntry
-> LLMUsageLog
-> CVParsedData or MatchResult explanation

LLM output supports the system. It does not own the final score.

24. Data Flow Acceptance Criteria

Data Flow / Sequence Diagrams v1 is accepted when:

Signup/onboarding flow is clear.
France Travail ingestion flow is clear.
Job normalization flow is clear.
Job skill extraction flow is clear.
Public job search flow is local database only.
Quick match flow is anonymous and LLM-free.
CV upload/parsing flow is async and privacy-safe.
Full match flow uses deterministic scoring.
LLM explanation flow cannot alter score.
Recommendation flow is precomputed and stored.
Weekly digest is opt-in and idempotent.
Account deletion flow removes private candidate data.
Admin actions reuse services.
Health check flow is defined.

25. Final Flow Decision

TuniTech Abroad uses this operational pattern:

Synchronous Django views for user interaction.
Celery workers for heavy processing.
PostgreSQL as source of truth.
Redis for queues and short-lived state.
France Travail API only in ingestion/revalidation services.
OpenRouter only through LLM service layer.
Email provider only through notification service layer.

This keeps the system buildable by one developer while keeping enough separation for future growth.

26. Next Document

After this document, the next planning document is:

Page + URL + View Map v1

It should define:

public URLs
dashboard URLs
admin URLs
view names
template paths
HTMX partials
auth requirements

service calls per view
page-level acceptance criteria